

Relatório de Validação de Rastreabilidade — Projeto Quântica Comercial

Framework LangNet · Validação ponta-a-ponta da cadeia de artefatos **Data:** 20/07/2026 **Projeto:** Comercial Quântica (b55ef718-0073-44d4-b279-11df89403e92) **Documento-fonte:** 20260615_031035_quantica-comercial-brief.md

Nota metodológica: a Quântica Comercial é a **instância de teste** do LangNet — não é o produto. Este relatório valida se os artefatos que o pipeline produziu para ela estão encadeados e coerentes, e serve de prova de campo para as correções de rastreabilidade implementadas nesta sessão.

1. Sumário executivo

O pipeline da Quântica tem **11 etapas** encadeadas, do documento inicial ao código Python. A validação percorreu cada etapa lendo, no banco langnet , **qual sessão e qual versão** de cada fonte originou cada artefato.

Veredito: a **espinha dorsal do pipeline (etapas 0–8)** está **coerente** — todos os artefatos, do documento à tasks.yaml , apontam para a **mesma Especificação atual** (fbc45992 , 06/jul) e para a **mesma cadeia Agent-Task Spec** (3c69a8a8). As **3 etapas finais (Sequência de Tarefas, Rede de Petri, Geração de Código)** apresentam incoerências de encadeamento, todas **diagnosticadas com precisão** neste relatório (§4) — duas delas com **correção retroativa bloqueada** por restrições de ambiente (§5), o que exige uma decisão do usuário.

Em paralelo, foram implementadas, testadas e versionadas **4 correções estruturais** no código do LangNet (§6) que passam a **gravar a proveniência (sessão + versão de cada fonte)** em toda geração futura — eliminando, dali em diante, a causa-raiz das incoerências encontradas.

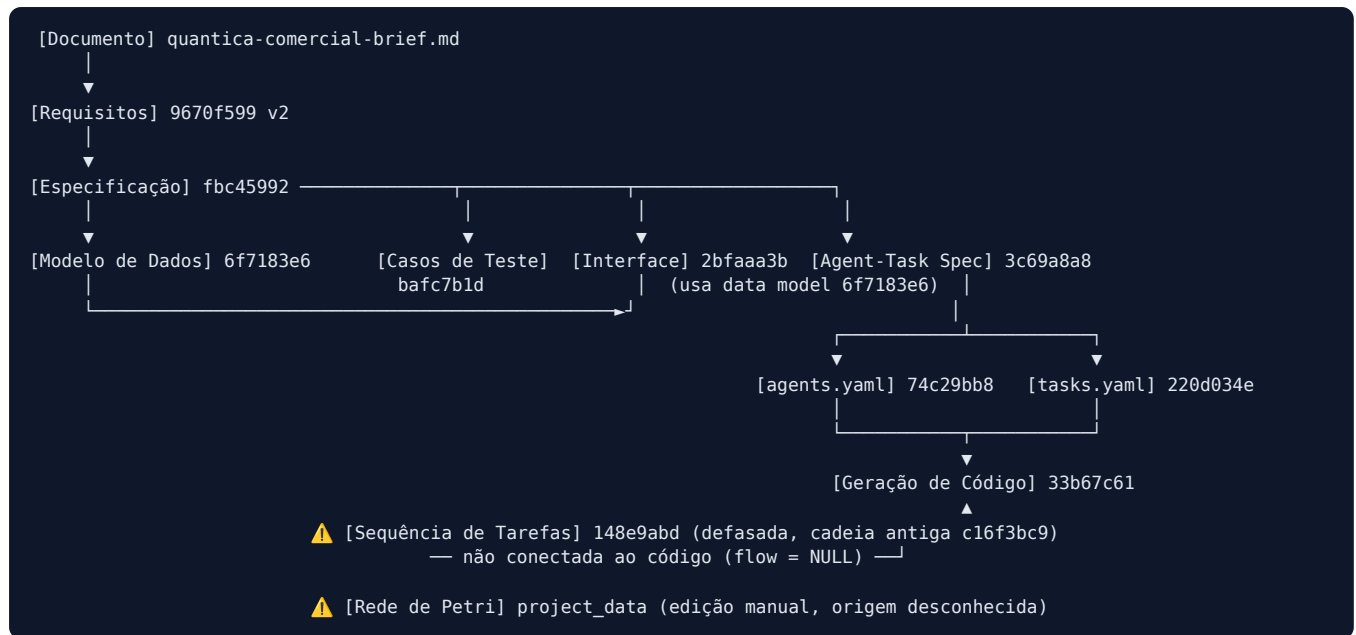
2. A cadeia de rastreabilidade da Quântica

Cada linha mostra a **sessão mais recente e útil** de cada etapa e a **origem que ela declara**.

#	Etapa	Sessão atual	Data	Origem declarada (sessão)	Versão gravada?	Coerência
0	Documento	quantica-comercial-brief.md	15/jun	— (upload)	—	raiz
1	Requisitos	9670f599 v2	06/jul	documento	✔ v2	✔
2	Especificação	fbc45992	06/jul	requisitos 9670f599 v2	✔ v2	✔
3	Modelo de Dados	6f7183e6	06/jul	spec fbc45992	⚠ NULL (pré-fix)	✔
4	Casos de Teste	bafc7b1d (aprovado)	18/jul	spec fbc45992	✔ v1 ¹	✔
5	Interface (ui_spec)	2bfaaa3b	17/jul	spec fbc45992 + data model 6f7183e6	⚠ NULL (pré-fix)	✔
6	Agent-Task Spec	3c69a8a8	13/jul	spec fbc45992 v1	✔ v1	✔
7	agents.yaml	74c29bb8	13/jul	agent_task_spec 3c69a8a8 v1	✔ v1	✔
8	tasks.yaml	220d034e	16/jul	agent_task_spec 3c69a8a8 v1	✔ v1	✔
9	Sequência de Tarefas	148e9abd	03/jul	spec c16f3bc9 (antiga) + ats 767146de (antigo) + tasks 1dba3682 (antigo)	⚠ NULL	⚠ DEFASADA
10	Rede de Petri	project_data (vh v1/v2)	19/jul	nenhuma (2 versões manual_edit)	⚠ NULL	⚠ SEM PROVENIÊNCIA
11	Geração de Código	33b67c61	17/jul	agents 74c29bb8 ✔ + tasks 220d034e ✔ + agent_task_spec 3c69a8a8 ✔ + flow NULL	⚠ NULL (pré-fix)	⚠ NÃO USOU A SEQUÊNCIA

¹ A sessão de Casos de Teste `cd483e4c` (draft, 20/jul) foi gerada durante o **teste da correção Etapa 1** desta sessão e **gravou corretamente** `specification_version = 1` — primeira prova de campo do novo mecanismo de proveniência.

2.1 Representação da cadeia



3. O que está coerente (etapas 0–8)

- **Uma única Especificação corrente.** Modelo de Dados, Casos de Teste, Interface e Agent-Task Spec apontam todos para `fbc45992` — não há divergência de spec entre os consumidores diretos.
- **A Interface amarra as duas fontes certas:** spec `fbc45992` e modelo de dados `6f7183e6` (ambos correntes) — a tela de negócio foi desenhada sobre o schema atual.
- **Cadeia YAML consistente:** Agent-Task Spec `3c69a8a8` → `agents.yaml` `74c29bb8` e `tasks.yaml` `220d034e`, ambos declarando `agent_task_spec_version = 1`.
- **A Geração de Código usa o topo correto da cadeia:** `agents.yaml`, `tasks.yaml` e `agent_task_spec` correntes.

4. Incoerências encontradas

4.1 [ALTA] Sequência de Tarefas defasada

A última Sequência de Tarefas **completa** (`148e9abd` , **03/jul**) foi gerada de uma cadeia **anterior**: spec `c16f3bc9` (a antiga, não a `fbc45992`), agent_task_spec `767146de` (antigo, não `3c69a8a8`) e tasks.yaml `1dba3682` (antigo, não `220d034e`). Ela **antecede** o Agent-Task Spec/agents/tasks atuais (13–16/jul) — logo, descreve um fluxo de execução de uma versão anterior do sistema.

4.2 [ALTA] Código não incorporou a Sequência de Tarefas

A sessão de código corrente (`33b67c61`) tem `task_execution_flow_session_id = NULL` — foi gerada **sem** a Sequência de Tarefas. O executor de Petri foi montado a partir de agents/tasks/petri, mas o **fluxo de execução planejado** (ordem, paralelismo) não entrou como insumo.

4.3 [MÉDIA] Rede de Petri sem proveniência

A Petri vive em `projects.project_data` e tem 2 versões no histórico, ambas `manual_edit` (19/jul), **sem vínculo** com agents.yaml/tasks.yaml/Sequência que a originaram. Não é possível, retroativamente, saber de qual versão das fontes ela derivou.

4.4 [BAIXA] Versões NULL em artefatos anteriores ao fix

Modelo de Dados (`6f7183e6`), Interface (`2bfaaa3b`) e Código (`33b67c61`) têm as colunas de versão da origem em NULL — foram gerados **antes** das correções desta sessão. O **vínculo de sessão** existe e está correto; falta apenas o número da versão, que passa a

ser gravado dali em diante.

5. Correção das incoerências — status e bloqueios

A instrução foi "se não estiverem coerentes, corrija". A correção natural é **regenerar** os artefatos da cauda a partir da cadeia atual. Isso esbarra em dois obstáculos reais:

5.1 Regenerar a Sequência de Tarefas — BLOQUEADO pelo contexto do LLM

Tentativa realizada nesta sessão (sessão `d32dea7e`): **falhou** com `Error 400 - Context size has been exceeded`. As três fontes somam **~33,6K tokens** de entrada (Especificação 20,7K + Agent-Task Spec 9,6K + tasks.yaml 3,2K), acima da janela do **qwen2.5-coder-32b** carregado no LM Studio (~32K). É exatamente o cenário anti-timeout já registrado na memória do projeto. **Opções (decisão do usuário)**: (a) carregar o modelo no LM Studio com janela maior (≥48K); (b) dividir a task de geração do fluxo em subtasks menores (padrão já usado no pipeline de especificação); (c) usar LLM de nuvem — **descartado** por decisão prévia do usuário.

5.2 Regenerar a Rede de Petri — RISCO de perda de edições manuais

A Petri corrente tem **2 versões** `manual_edit` (trabalho manual do usuário). Uma nova geração via LLM **sobrescreveria** `project_data`. Regenerar só para preencher colunas de proveniência é um mau negócio. **Decisão do usuário**: manter a Petri atual (proveniência retroativa fica indisponível, mas a próxima geração via UI já grava tudo — Etapa 3) **ou** aceitar regenerar e perder as edições manuais.

5.3 O que já foi corrigido de forma definitiva

A causa-raiz — **o pipeline não gravava a versão das fontes** — foi corrigida no código do LangNet (§6). Toda geração **futura** de qualquer etapa passa a gravar a proveniência completa; a sessão de Casos de Teste `cd483e4c` (§2, nota 1) já comprova isso em campo.

6. Correções estruturais implementadas (código do LangNet)

Quatro etapas, cada uma com **backup do banco + commit + push + teste** antes de avançar:

Etapas	Commit	Migration	O que passou a ser gravado	Teste
1 — Consumidores da Especificação	<code>8f1b143</code>	023–025	<code>specification_version</code> em Modelo de Dados, Casos de Teste e Interface (+ <code>data_model_version</code> na Interface)	Casos de Teste nova gravou <code>specification_version=1</code>
2 — Sequência de Tarefas	<code>a08bc4c</code>	026	<code>specification_version</code> , <code>agent_task_spec_version</code> , <code>tasks_yaml_version</code>	Geração gravou <code>1/1/1</code>
3 — Rede de Petri	<code>ea23ab4</code>	027	sessão+versão de agents.yaml, tasks.yaml e Sequência de Tarefas	Insert real gravou agents v1 + tasks v1
4 — Geração de Código	<code>78bd9d1</code>	028	versão das 4 fontes + versão da Petri usada + sessão/versão do ui_spec	Insert real: agents v1, tasks v1, spec v1, petri v2, ui_spec capturado

Padrão comum: cada geração lê `MAX(version)` da `*_version_history` da fonte e carimba na linha do artefato. Camada **aditiva e tolerante a falha** — versionamento nunca quebra a geração.

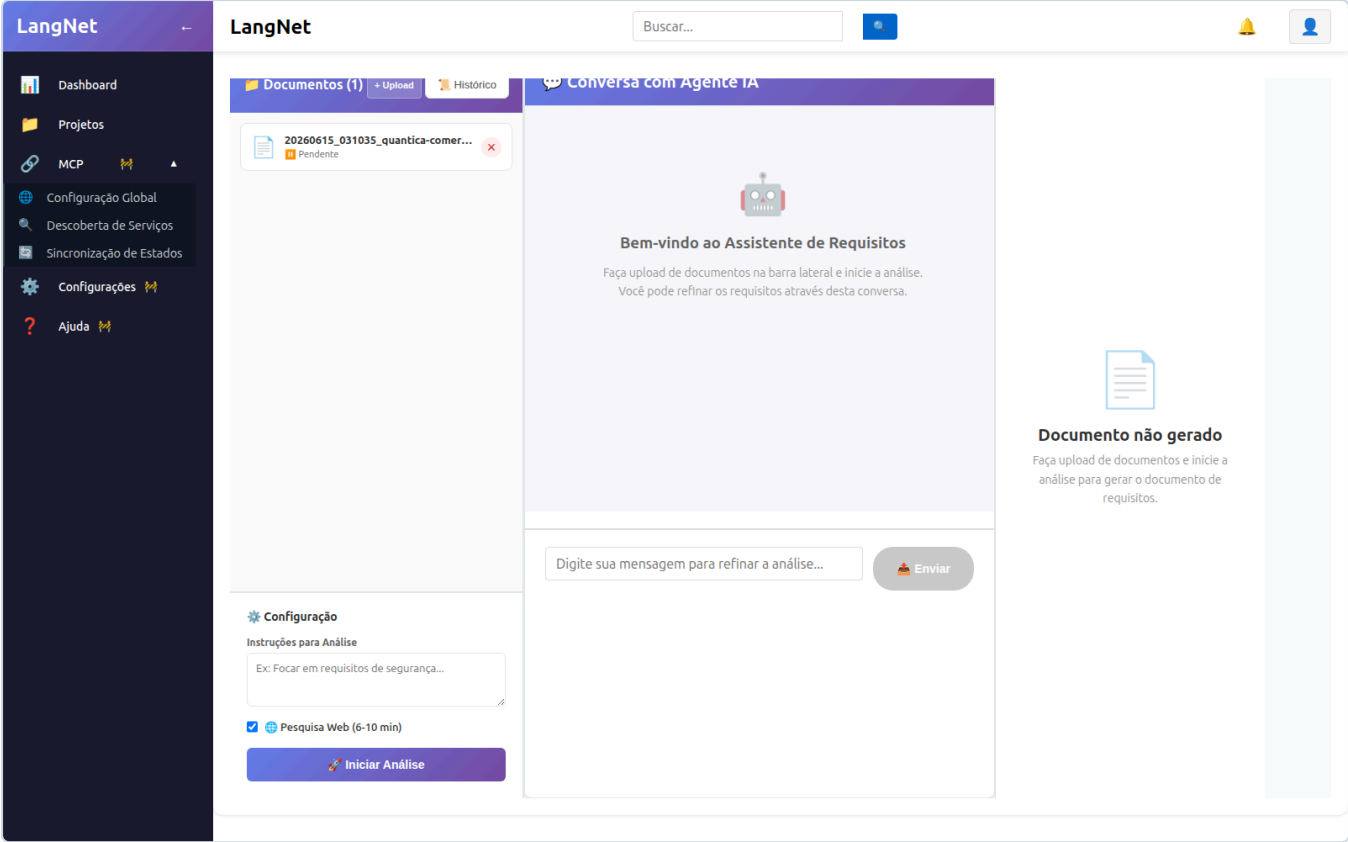
7. Recomendações

- Decidir sobre a Sequência de Tarefas (§5.1)** — sem isso, a cauda do pipeline permanece descolada da cadeia atual. Recomendação: aumentar a janela do LM Studio para ≥48K e regenerar; isso reconecta Sequência → Petri → Código com proveniência completa (validando as Etapas 2–4 em geração real de ponta a ponta).
- Após a Sequência, regenerar o Código** passando `task_execution_flow_session_id` (corrige §4.2).
- Ao evoluir a Petri**, fazê-lo pela UI (Gerar Rede) para nascer com proveniência (Etapa 3), preservando a decisão do usuário sobre as edições manuais atuais (§5.2).
- Próxima feature de rastreabilidade**: exibir na UI o rótulo "gerado de: {fonte} v{n}" em cada etapa, consumindo as novas colunas — fecha o ciclo tornando a proveniência visível ao usuário.

Anexo — Captura de telas (Quântica Comercial, 20/07/2026)

Telas capturadas em headless (Playwright, viewport 1600×1000) na instância local. Algumas etapas (Especificação, Sequência de Tarefas) exibem o estado inicial pois a página não auto-carrega a última sessão ao ser aberta direto pela URL — o artefato existe no banco (ver §2). A tela de **Modelo de Dados** já mostra a proveniência ("Origem: Especificação fbc45992" + seletor de versão) diretamente na UI.

0. Documento-fonte



1. Especificação

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Docs Complementares (1)

20260615_031035_quantica-comer...

Complementar

Conversa com Agente IA

Bem-vindo ao Assistente de Requisitos

Faça upload de documentos na barra lateral e inicie a análise. Você pode refinar os requisitos através desta conversa.

Digite sua mensagem...

Analisar

Refinar

Configuração

Instruções Adicionais

Ex: Focar em requisitos de segurança, detalhar casos de uso...

Wireframe dos Casos de Uso

ASCII Art

PlantUML Salt

Gerar Especificação

Revisar Especificação

Especificação não gerada

1. Selecione um documento de requisitos (Requisitos)

2. Opcionalmente adicione docs complementares

3. Clique em "Gerar Especificação"

2. Modelo de Dados (mostra banner "Origem: Especificação fbc45992" + dropdown Spec v2)

LangNet

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Modelo de Dados

Gera schema SQL, models.py (SQLAlchemy+Pydantic) e migrations Alembic a partir da especificação. Extraí entidades e relacionamentos automaticamente.

Rascunho Versão v1 DBMS MYSQL 19 tabelas

Validação (score 75/100) — 2 problemas

Entidades

personas

problemas

canais

3. Casos de Teste & Validação

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Casos de Teste & Validação

Casos de teste derivados dos casos de uso pela técnica do Grafo de Causa-Efeito (causas = ações do ator, efeitos = respostas do sistema). Cada coluna da tabela de decisão é um caso de teste.

Casos de Uso

Histórico

Spec v2 (fbc45992...)

Origem: Especificação fbc45992...

Configuração

Instruções Adicionais

Gerar casos de teste

Revisar

Refinar com o agente

DRAFT

Versão v1

1 casos de uso

4 casos de teste

Documento

Aprovar

UC-001

Cadastrar Persona-Alvo

4 casos

UC-001 — Cadastrar Persona-Alvo

Ator: CEO Objetivo: Criar um novo perfil do cliente-alvo com características específicas. 2 causas · 5 efeitos · 4 casos

Tabela de Decisão

	C1	C2	C3	C4
c1	1	1	0	X
c2	X	X	X	1
e1	X	X	X	X
e2	1	1	0	X
e3	1	1	0	X
e4	0	0	1	X
e5	X	X	X	1

1 = verdadeiro · 0 = falso · X = indiferente (don't-care). Efeito em verde = ativado.

Casos de Teste (4)

TC-UC-001-01

ENTRADAS (AÇÕES DO ATOR)

campos obrigatórios preenchidos

RESPOSTA ESPERADA DO SISTEMA

campos preenchidos conforme entrada do usuário

TC-UC-001-02

ENTRADAS (AÇÕES DO ATOR)

campos obrigatórios preenchidos

4. Interface (ui_spec)

LangNet

Buscar...

Interface & Protótipo

Especificação e protótipo de interface gerados a partir da Especificação Funcional (casos de uso + wireframes) e do Modelo de Dados. Cada tela vira um mockup PNG refinável por chat.

Telas

Histórico

Spec v2 (fbc45992...)

Origem: Especificação fbc45992...

Configuração

Instruções Adicionais

Gerar UI Spec

Refinar com o agente

DRAFT

Versão v2

18 telas

Aprovar

Formulário de Cadastro de Persona

UC-001 - form - personas

Calendário Mensal Gerado Automaticamente

UC-002 - detail

Editar Calendário Mensal

UC-003 - table - posts

Aprovação do Calendário Mensal

UC-004 - dashboard - calendario_mensal

Sugestões Semanais de Temas

UC-005 - detail

Geração Automática de Conteúdo

UC-006 - detail

Verificação de Fatos no Conteúdo Gerado

UC-007 - detail

Revisão do Conteúdo Gerado

UC-008 - form - posts

Agendamento de Publicações

UC-009 - dashboard

Publicação Automática de Conteúdo

UC-010 - detail

Coleta de Métricas de Engajamento

UC-011 - dashboard - metricas_engajamento

Classificação Automática de Comentários

UC-012 - detail

Geração Automática de Respostas a Comentários

Formulário de Cadastro de Persona

Nome do Produto

Dashboard

Personas

Contatos

Novo Persona

Crie um novo perfil do cliente-alvo com características específicas.

Nome

Descrição

Problemas

Canais

Gatilhos de Compra

Objções

Palavras-chave

Telefone

Salvar

Cancelar

Componentes

5. Agent-Task Spec

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Configuração

Histórico

Especificação

Selecione uma especificação funcional

Selecione uma especificação funcional

Configuração

Nível de Detalhamento

Balancedo

Frameworks Suportados

CrewAI

AutoGen

LangChain

Custom

Instruções Adicionais

Ex: focar em cenários de erro, detalhar validações...

Gerar Agentes & Tarefas

Revisar

Conversa com Agente IA

Bem-vindo ao Assistente de Requisitos

Faça upload de documentos na barra lateral e inicie a análise. Você pode refinar os requisitos através desta conversa.

Digite sua mensagem para refinar a análise...

Enviar

Nenhum Documento Gerado

Selecione uma especificação funcional e clique em "Gerar Agentes & Tarefas"

6. agents.yaml

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Agentes

2 agentes

Especificador de Agentes

+ Novo Agente

Exportar YAML

Buscar agentes...

Todos os Status

customer_service_agent

ATIVO

Role: Especialista em atendimento ao cliente responsável por lidar com consultas

Goal: Entender as necessidades do cliente e fornecer soluções apropriadas

Tools: knowledge_base_tool ticket_creation_tool customer_history_tool

Verbose

Editar Excluir

technical_support_agent

ATIVO

Role: Especialista em suporte técnico para questões complexas

Goal: Resolver problemas técnicos e fornecer soluções eficazes

Tools: diagnostic_tool remote_access_tool knowledge_base_tool

Verbose Delegation Code Exec

Editar Excluir

7. tasks.yaml

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Tarefas 2 tarefas

Especificador de Tarefas

+ Nova Tarefa

Exportar YAML

Buscar tarefas...

Todos os Status

Todos os Agentes

process_customer_query_task

ATIVA

Descrição:
Analisar e processar consulta de cliente identificando a natureza e urgência

Saída Esperada:
Classificação estruturada da consulta com prioridade e roteamento adequado

Agente Responsável:
customer_service_agent

Ferramentas:
TextAnalysisTool ClassificationModel

II

analyze_sentiment_task

ATIVA

Descrição:
Analisar o sentimento da mensagem do cliente para determinar o nível de satisfação

Saída Esperada:
Score de sentimento com análise detalhada e recomendações de ação

Agente Responsável:
customer_service_agent

Contexto:
process_customer_query_task

Ferramentas:
SentimentAnalyzer EmotionDetector

Assíncrono

JSON

Arquivo

II

8. Sequência de Tarefas (defasada)

LangNet

Buscar...

Docs Complementares (1)

Histórico

Upload

Specs & Docs

Configuração

20260615_031035_quantica...

Completar

Selecione os 3 documentos obrigatórios primeiro (Specs & Docs)

Instruções Adicionais

Ex: focar em cenários de erro, detalhar validações...

Gerar Sequência de Tarefas

Revisar

Conversa com Agente IA

Bem-vindo ao Assistente de Requisitos

Faça upload de documentos na barra lateral e inicie a análise. Você pode refinar os requisitos através desta conversa.

Digite sua mensagem...

Analisar

Refinar

Sequência de Tarefas não gerada

1. Selecione os documentos base (Specs & Docs):

• Especificação Funcional

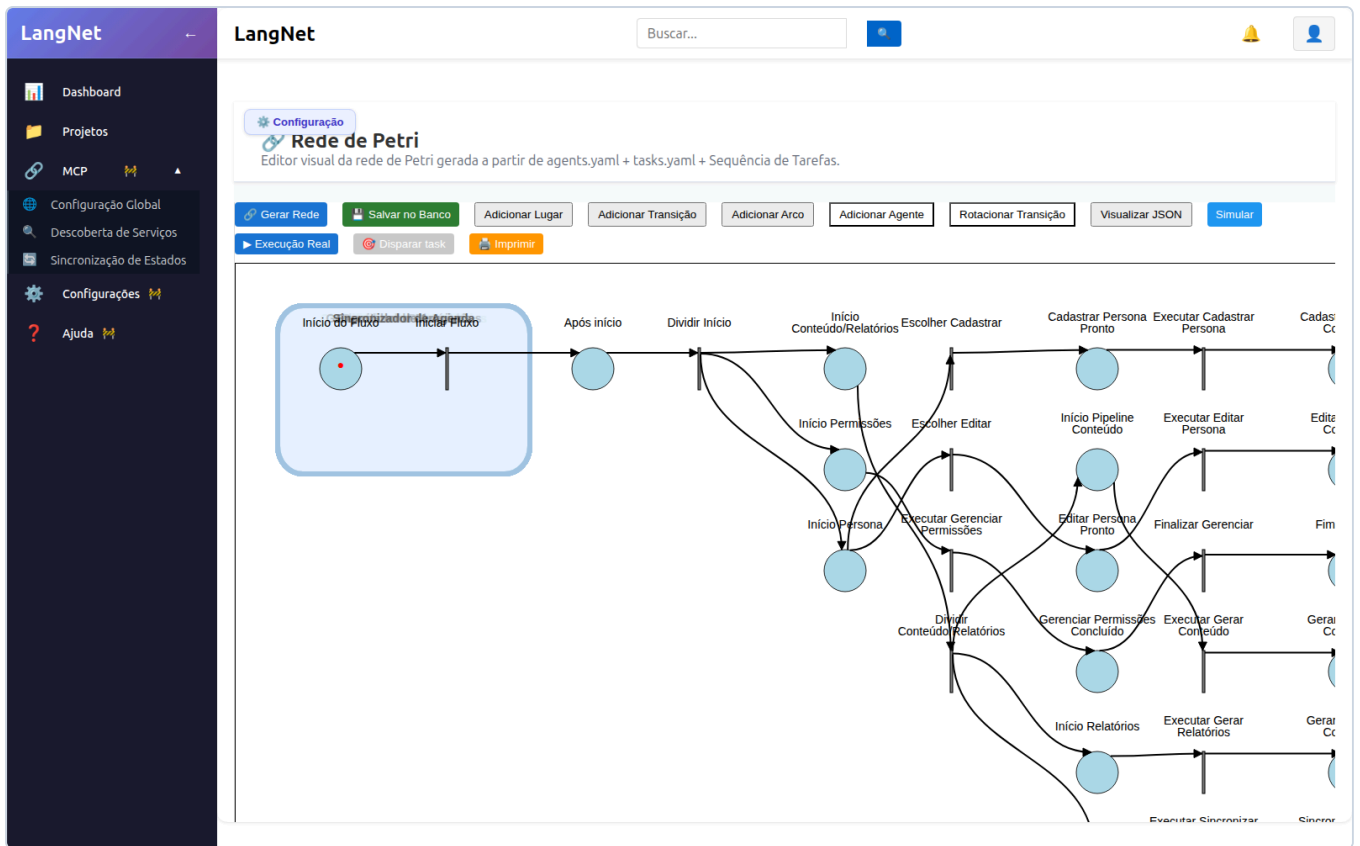
• Agent/Task Spec (especificação de agentes e tarefas)

• Tasks YAML (configuração de tarefas)

2. Opcionalmente adicione instruções customizadas

3. Clique em Gerar Sequência de Tarefas

9. Rede de Petri



10. Geração de Código (37 sessões)

LangNet

Buscar...

Geração de Código
Geração do sistema agêntico Python (agents.yaml, tasks.yaml, tools.py, adapters.py, petri_net.json...) a partir das especificações. Refine com o agente, corrija warnings e execute o servidor.

Salvar Arquivo Baixar ZIP Executar

Geração Histórico Nova geração

ARQUIVOS (0)
Nenhum arquivo. Clique em Nova geração.

Selecione um arquivo à esquerda, ou clique em Nova geração para começar.

Configuração

SESSÕES (37)

- code_gen_20260717_201202
v1 - 55 arquivos - completed
- code_gen_20260717_144213
v1 - 55 arquivos - completed
- code_gen_20260717_050623
v1 - 55 arquivos - completed
- code_gen_20260717_033128
v1 - 55 arquivos - completed

Instruções Adicionais

Ex: focar em cenários de erro, detalhar validações...

Gerar Código Refinar com o agente